

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ЛАБОРАТОРНАЯ РАБОТА 6

Работа с БД в СУБД MongoDB

Обучающийся Сафонова Арина Дмитриевна

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Задачи:

1. Пересоздать коллекцию unicorns и заполнить её исходными документами из условия лабораторной работы.
2. Проверить возможность создания уникального индекса по полю name для коллекции unicorns.
3. Получить информацию обо всех индексах коллекции unicorns с помощью команды getIndexes.
4. Удалить пользовательский индекс name_1 и оставить только стандартный индекс id.
5. Попыаться удалить системный индекс id и убедиться, что MongoDB не позволяет удалить обязательный индекс идентификатора.
6. Создать объёмную коллекцию numbers, содержащую 100000 документов с полем value.
7. Выполнить запрос на выбор последних четырёх документов из коллекции numbers.
8. Проанализировать план выполнения запроса без индекса с помощью метода explain с параметром executionStats.
9. Определить время выполнения запроса без индекса по значению executionTimeMillis.
10. Создать индекс value_1 по полю value для коллекции numbers.
11. Получить информацию обо всех индексах коллекции numbers.
12. Повторно выполнить запрос на выбор последних четырёх документов после создания индекса.
13. Проанализировать план выполнения запроса с установленным индексом.
14. Сравнить время выполнения запроса без индекса и с индексом.
15. Сделать вывод о том, что использование индексов повышает эффективность выполнения запросов, так как MongoDB просматривает меньше документов и быстрее возвращает результат.

Практическое задание:

Практическое задание 2.1.1:

1. *Создайте базу данных learn.*

```
test> use learn
switched to db learn
learn> █
```

2. *Заполнение коллекции единорогов unicorns:*

```
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2de2af02878a3ff109f928') }
}
learn> █
```

3. Использование второго способа, вставки в коллекцию единорогов документ:

learn> `document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}`

```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
|
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
```

learn> `db.unicorns.insertOne(document)`

```
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a2de3fb02878a3ff109f929')
}
```

4. Проверьте содержимое коллекции с помощью метода find.

learn> `db.unicorns.find()`

```

learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f91e'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f91f'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f920'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f921'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f922'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f923'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f926'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f927'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f928'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a2de3fb02878a3ff109f929'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> █

```

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a2de3fb02878a3ff109f929'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f91e'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f927'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f921'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f920'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

Через `findOne`

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a2de2af02878a3ff109f91f'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> █
```

Через limit :

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f91f'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a2de3fb02878a3ff109f929'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f91e'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f927'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f921'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f920'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural: -1})
[
```

```

    _id: ObjectId('6a2de3fb02878a3ff109f929'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f928'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f927'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f926'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f923'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f922'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f921'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f920'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f91f'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },

```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
```

```
[learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
[learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires:{$exists:false}})
```

```
learn> db.unicorns.find({vampires:{$exists:false}})
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f928'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB. ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
learn> db.towns.insertMany([{"name": "Punxsutawney ",
| populatiuon: 6200,
| last_sensus: ISODate("2008-01-31"),
| famous_for: [""],
| mayor: {
|   name: "Jim Wehrle"
| }},
| {"name": "New York",
| populatiuon: 22200000,
| last_sensus: ISODate("2009-07-31"),
| famous_for: ["status of liberty", "food"],
| mayor: {
|   name: "Michael Bloomberg",
| party: "I"}},
| {"name": "Portland",
| populatiuon: 528000,
| last_sensus: ISODate("2009-07-20"),
| famous_for: ["beer", "food"],
| mayor: {
|   name: "Sam Adams",
| party: "D"}}
| ])
```

```
learn> db.towns.insertMany([
  {name: "Punxsutawney ",
  populatiuon: 6200,
  last_sensus: ISODate("2008-01-31"),
  famous_for: [""],
  mayor: {
    name: "Jim Wehrle"
  }},
  {name: "New York",
  populatiuon: 22200000,
  last_sensus: ISODate("2009-07-31"),
  famous_for: ["status of liberty", "food"],
  mayor: {
    name: "Michael Bloomberg",
    party: "I"}}},
  {name: "Portland",
  populatiuon: 528000,
  last_sensus: ISODate("2009-07-20"),
  famous_for: ["beer", "food"],
  mayor: {
    name: "Sam Adams",
    party: "D"}}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a2e80002878a3ff109f92a'),
    '1': ObjectId('6a2e80002878a3ff109f92b'),
    '2': ObjectId('6a2e80002878a3ff109f92c')
  }
}
learn>
```

2. Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn>
```

3. Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

```
[learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Практическое задание 3.1.2:

3. Сформировать функцию для вывода списка самцов единорогов.

```
fn = function() { return this.gender == 'm'; }
```

```
[learn> fn = function() { return this.gender == 'm'; }  
[Function: fn]  
learn> 
```

4. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
[learn> var cursor = db.unicorns.find({$where: fn}).sort({name: 1}).limit(2); null;  
null  
learn> 
```

5. Вывести результат, используя *forEach*.

```
learn> cursor.forEach(function(obj){ print(obj.name); })
```

```
learn> cursor.forEach(function(obj){ print(obj.name); })  
Dunx  
Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
```

```
[learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()  
2  
learn> 
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]  
learn> 
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}]])
```

```
[learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}]])  
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]  
learn> ]
```

Практическое задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],  
weight: 340, gender: 'm'})
```

```
TypeError: db.unicorns.save is not a function  
learn> db.unicorns.save({name: 'Barney', loves: ['grape'],  
| weight: 340, gender: 'm'})  
|  
TypeError: db.unicorns.save is not a function
```

Поэтому save был заменен на insertOne

```
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
```

```
[learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a2e8e4d02878a3ff109f92e')  
}]
```

2. *Проверить содержимое коллекции unicorns.*

```
db.unicorns.find()
```



```

    weight: 675,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f922'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f923'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f926'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,

```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэббул.

```
db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
```

```

[learn> db.unicorns.updateOne({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> █

```

2. Проверить содержимое коллекции unicorns.

```

{
  _id: ObjectId('6a2de2af02878a3ff109f924'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a2de2af02878a3ff109f925'),
  name: 'Raleigh',
  loves: [ 'redbull' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a2de2af02878a3ff109f926'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a2de2af02878a3ff109f927'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a2de2af02878a3ff109f928'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
}
}

```

Практическое задание 3.3.4:

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции unicorns.*

```
learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
```

```
db.unicorns.find()
```

```

[learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9,
  modifiedCount: 9,
  upsertedCount: 0
}
[learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f91e'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f91f'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f920'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f921'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f922'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f923'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f924'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44
  },
  {
    _id: ObjectId('6a2de2af02878a3ff109f925'),
    name: 'Raleigh'
  }
]

```

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции `towns`.

```
db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": 1}})
```

```
learn> db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

learn> db.towns.find()

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a2e80002878a3ff109f92a'),
    name: 'Punxsutawney ',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a2e80002878a3ff109f92b'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a2e80002878a3ff109f92c'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
2. Проверить содержимое коллекции *unicorns*.

```
db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
```

```
db.unicorns.find({name: 'Pilot'})
```

```
learn> db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f927'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> 
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
```

```
learn> db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a2de2af02878a3ff109f91f'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> 
```

```
db.unicorns.find({name: 'Aurora'})
```

Практическое задание 3.4.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
  popujatiun: 6200,
  last_sensus: ISODate("2008-01-31"),
  famous_for: ["phil the groundhog"],
  mayor: {
    name: "Jim Wehrle"
  }}
}
```

```
{name: "New York",
  popujatiun: 22200000,
  last_sensus: ISODate("2009-07-31"),
}
```

```
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

Удаление старых данных

```
learn> db.towns.drop()
true
```

Создание новой

```
learn> db.towns.insertMany([
| {name: "Punxsutawney ",
|   popujatiuon: 6200,
|   last_sensus: ISODate("2008-01-31"),
|   famous_for: ["phil the groundhog"],
|   mayor: {
|     name: "Jim Wehrle"
|   }},
| {name: "New York",
|   popujatiuon: 22200000,
|   last_sensus: ISODate("2009-07-31"),
|   famous_for: ["status of liberty", "food"],
|   mayor: {
|     name: "Michael Bloomberg",
|     party: "I"}},
| {name: "Portland",
|   popujatiuon: 528000,
|   last_sensus: ISODate("2009-07-20"),
|   famous_for: ["beer", "food"],
|   mayor: {
|     name: "Sam Adams",
|     party: "D"}}
| ])
{
```

```

[learn> db.towns.drop()
true
[learn> db.towns.insertMany([{name: "Punxsutawney ",
| popujatiuon: 6200,
| last_sensus: ISODate("2008-01-31"),
| famous_for: ["phil the groundhog"],
| mayor: {
|   name: "Jim Wehrle"
| }},
| {name: "New York",
| popujatiuon: 22200000,
| last_sensus: ISODate("2009-07-31"),
| famous_for: ["status of liberty", "food"],
| mayor: {
|   name: "Michael Bloomberg",
| party: "I"}},
| {name: "Portland",
| popujatiuon: 528000,
| last_sensus: ISODate("2009-07-20"),
| famous_for: ["beer", "food"],
| mayor: {
|   name: "Sam Adams",
| party: "D"}}
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a2e994602878a3ff109f92f'),
    '1': ObjectId('6a2e994602878a3ff109f930'),
    '2': ObjectId('6a2e994602878a3ff109f931')
  }
}
[learn> █

```

6. Удалите документы с беспартийными мэрами.
7. Проверьте содержание коллекции.

```

[learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
[learn> db.towns.find()
[
  {
    _id: ObjectId('6a2e994602878a3ff109f930'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a2e994602878a3ff109f931'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
[learn> █

```

```

learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }

learn> db.towns.find()

```

8. Очистите коллекцию.
9. Просмотрите список доступных коллекций.

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> █
```

```
db.towns.deleteMany({})
show collections
```

Практическое задание 4.1.1:

7. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.zones.insertMany([{_id: "magic_forest", full_name: "Magic Forest", description: "Unicorn habitat in a magical forest"}, {_id: "mountains", full_name: "Mountains", description: "Unicorn habitat in the mountains"}, {_id: "forest_lakes", full_name: "Forest Lakes", description: "Unicorn habitat near forest lakes"}])
```

8. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```
learn> db.unicorns.update({name: "Aurora"}, {$set:{zone:{$ref:"zones", $id: "magic_forest"}}})
```

```
learn> db.unicorns.update({name: "Unicrom"}, {$set:{zone:{$ref:"zones", $id: "mountains"}}})
```

```
learn> db.unicorns.update({name: "Nimue"}, {$set:{zone:{$ref:"zones", $id: "forest_lakes"}}})
```

```
learn> db.unicorns.update({name: "Aurora"}, {$set:{zone:{$ref:"zones", $id: "magic_forest"}}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: "Unicrom"}, {$set:{zone:{$ref:"zones", $id: "mountains"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: "Nimue"}, {$set:{zone:{$ref:"zones", $id: "forest_lakes"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> █
```

9. Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find({name: {$in: ["Aurora", "Unicrom", "Nimue"]}, {name: 1, zone: 1, _id: 0}})
[
  { name: 'Aurora', zone: DBRef('zones', 'magic_forest') },
  { name: 'Unicrom', zone: DBRef('zones', 'mountains') },
  { name: 'Nimue', zone: DBRef('zones', 'forest_lakes') }
]
learn> █
```

```
learn> db.unicorns.find({name: {$in: ["Aurora", "Unicrom", "Nimue"]}, {name: 1, zone: 1, _id: 0}})
```

4.2 НАСТРОЙКА ИНДЕКСОВ

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.drop()
true
```

Создание с обновленными данными

```
learn> db.unicorns.insertMany([
  {name: 'Horny', dob: new Date(1992,2,13,7,47), loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0), loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10), loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Roooooodles', dob: new Date(1979, 7, 18, 18, 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1), loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', dob: new Date(1998, 2, 7, 8, 30), loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', dob: new Date(1997, 6, 1, 10, 42), loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57), loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leila', dob: new Date(2001, 9, 8, 14, 53), loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3), loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', dob: new Date(1999, 11, 20, 16, 15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'},
  {name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18), loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a2ea1fe02878a3ff109f932'),
    '1': ObjectId('6a2ea1fe02878a3ff109f933'),
    '2': ObjectId('6a2ea1fe02878a3ff109f934'),
    '3': ObjectId('6a2ea1fe02878a3ff109f935'),
    '4': ObjectId('6a2ea1fe02878a3ff109f936'),
    '5': ObjectId('6a2ea1fe02878a3ff109f937'),
    '6': ObjectId('6a2ea1fe02878a3ff109f938'),
    '7': ObjectId('6a2ea1fe02878a3ff109f939'),
    '8': ObjectId('6a2ea1fe02878a3ff109f93a'),
    '9': ObjectId('6a2ea1fe02878a3ff109f93b'),
    '10': ObjectId('6a2ea1fe02878a3ff109f93c'),
    '11': ObjectId('6a2ea1fe02878a3ff109f93d')
  }
}
learn>
```

Корзина

```
db.unicorns.ensureIndex({"name": 1}, {"unique": true})
```

```
learn> db.unicorns.ensureIndex({"name" : 1}, {"unique" : true})
[ 'name_1' ]
learn>
```

Практическое задание 4.3.1:

1. Получите информацию о всех индексах коллекции `unicorns`.

```
learn> db.unicorns.getIndexes()
```

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

2. Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
```

learn>

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn>
```

3. Попробуйте удалить индекс для идентификатора.

learn> db.unicorns.dropIndex("_id_")

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:

learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}

```
[learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}

{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2ea65902878a3ff10b7fdd') }
}
```

2. Выберите последних четыре документа.

learn> db.numbers.find().sort({value: -1}).limit(4)

```
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a2ea65902878a3ff10b7fdd'), value: 99999 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fdc'), value: 99998 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fdb'), value: 99997 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fda'), value: 99996 }
]
learn>
```

При выполнении запроса MongoDB вернула 4 документа (nReturned: 4). Так как индекс по полю *value* ещё не создан, был выполнен полный просмотр коллекции (COLLSCAN), всего просмотрено 100000 документов (totalDocsExamined: 100000). После этого была выполнена сортировка (SORT). Время выполнения запроса составило 164 мс по параметру *executionTimeMillis*.

3. Создайте индекс для ключа *value*.

learn> db.numbers.ensureIndex({"value": 1})

```
[learn> db.numbers.ensureIndex({"value": 1})
[ 'value_1' ]
learn>
```

4. Получите информацию о всех индексах коллекции *numbers*.

learn> db.numbers.getIndexes()

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

5. Выполните запрос 2.

```
learn> db.numbers.find().sort({value: -1}).limit(4)
```

```
[
  { _id: ObjectId('6a2ea65902878a3ff10b7fdd'), value: 99999 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fdc'), value: 99998 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fdb'), value: 99997 },
  { _id: ObjectId('6a2ea65902878a3ff10b7fda'), value: 99996 }
]
learn> █
```

6. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

После создания индекса value_1 запрос стал выполняться с использованием индекса (IXSCAN). Время выполнения по параметру executionTimeMillis составило 25 мс. До создания индекса запрос выполнялся за 164 мс и использовал полный просмотр коллекции (COLLSCAN), поэтому индекс ускорил выполнение запроса.

```
> db.numbers.explain("executionStats").find({}).sort({value: -1}).limit(4)
```

7. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Запрос с индексом является более эффективным, потому что он выполнен быстрее: **25 мс вместо 164 мс**. Кроме того, без индекса MongoDB выполняла полный просмотр коллекции и обработала 100000 документов, а с индексом был использован индексный просмотр и обработано только 4 документа. Следовательно, индекс value1 значительно ускорил выполнение запроса.

Выводы

В ходе выполнения лабораторной работы были изучены основные операции работы с коллекциями, индексами и планами выполнения запросов в MongoDB.

Была пересоздана коллекция unicorns и заполнена исходными документами из условия лабораторной работы. Для коллекции был создан уникальный индекс по полю `name` с параметром unique: true. В результате было проверено, что уникальный индекс можно создать только в том случае, если значения индексируемого поля не повторяются.

Также была получена информация обо всех индексах коллекции `unicorns` с помощью команды getIndexes(). Было установлено, что MongoDB автоматически создаёт системный индекс _id_, который используется для идентификации документов. Дополнительно был удалён пользовательский индекс name_1, а при попытке удаления индекса _id_ было подтверждено, что системный индекс идентификатора удалить нельзя.

Во второй части работы была создана объёмная коллекция numbers, содержащая 100000 документов. Был выполнен запрос на выбор последних четырёх документов с сортировкой по полю value. С помощью метода explain("executionStats") был проанализирован план выполнения запроса до создания индекса и после его создания.

До создания индекса MongoDB выполняла полный просмотр коллекции ('COLLSCAN'), было просмотрено 100000 документов, а время выполнения запроса составило 164 мс. После создания индекса value_1 по полю value запрос стал выполняться с использованием индексного просмотра I, было просмотрено только 4 ключа индекса и 4 документа, а время выполнения уменьшилось до 25 мс.

Таким образом, было доказано, что использование индексов значительно повышает эффективность выполнения запросов. Запрос с индексом оказался более эффективным, так как выполнялся быстрее и требовал просмотра меньшего количества документов.